

Calcul Distribué : **Hadoop**

Ecole Nationale des Sciences Appliquées de Khouribga

Pr. SOUSSI Nassima

Plan du Cours

- I. Généralité sur les systèmes distribués
- II. Ecosystème Hadoop (*version 1, 2 et 3*)
 - Composants : HDFS, MapReduce et YARN
 - Architecture : Processus internes et déroulement de tâche sur le cluster Hadoop
 - Limitations et Avantages de chaque version

Introduction aux Systèmes Distribués



Systeme Distribué : Définition

- Un **système distribué** est un ensemble de nœuds (ordinateurs) autonomes qui collaborent pour atteindre un objectif commun.
- Les nœuds communiquent et coordonnent leurs actions en échangeant des messages sur un réseau.
- L'objectif principal est de rendre le système **transparent**, de sorte que les utilisateurs perçoivent l'ensemble des nœuds comme un seul système unifié, même si les composants sont physiquement séparés.

Systeme Distribué : Caractéristiques

- **Transparence** : Les utilisateurs n'ont pas besoin de connaître l'emplacement ou le nombre de nœuds.
- **Scalabilité** : La capacité à augmenter les ressources en ajoutant plus de nœuds sans perturber les services.
- **Tolérance aux pannes** : Le système doit continuer de fonctionner en cas de panne de certains nœuds.
- **Concurrence** : Plusieurs nœuds peuvent traiter des opérations simultanément, améliorant les performances globales.

Systeme Distribué : Architecture

1. Architecture Maitre-Esclave : Principe

C'est un modèle de système distribué où un nœud central, appelé le **maître**, contrôle un ou plusieurs **esclaves**.

- **Le maître** est responsable de la gestion des esclaves, de la distribution des tâches, de l'agrégation des résultats, et du maintien de la cohérence des données.
- **Les esclaves** exécutent les tâches qui leur sont assignées par le maître, souvent en parallèle pour améliorer les performances. Les esclaves envoient ensuite leurs résultats au maître, qui peut en faire la synthèse.

Systeme Distribué : Architecture

1. Architecture Maître-Esclave : Avantages

- ✓ **Simplicité de Coordination** : Le maître gère la répartition des tâches et peut surveiller les esclaves, simplifiant la coordination des tâches et des ressources.
- ✓ **Performances Élevées pour le Calcul Parallèle** : La division des tâches entre esclaves permet de traiter en parallèle, accélérant le traitement de données massives.
- ✓ **Fiabilité avec Réplication** : La redondance des données sur plusieurs esclaves offre une tolérance aux pannes, puisque le maître peut réaffecter les tâches en cas de défaillance d'un esclave.

Systeme Distribué : Architecture

1. Architecture Maître-Esclave : Inconvénients

- **Point de Défaillance Unique** : Le maître constitue un **point de défaillance unique**. Si le maître échoue, l'ensemble du système peut devenir inutilisable. Certains systèmes introduisent des mécanismes de basculement ou de duplication du maître pour pallier cette faiblesse.
- **Scalabilité Limitée** : Le maître doit gérer la communication et la coordination de tous les esclaves, ce qui limite la scalabilité. Plus il y a d'esclaves, plus le maître est sollicité, pouvant entraîner des blocages.

Systeme Distribué : Architecture

2. Architecture Peer-to-Peer (P2P)

- Repose sur un modèle décentralisé où chaque nœud du réseau, appelé "pair" ou "peer", agit à la fois en tant que client et serveur.
- Tous les nœuds sont égaux en termes de rôle et de responsabilités, ce qui contraste avec les modèles client-serveur traditionnels où un serveur central répond aux requêtes des clients.
- Cette approche permet de **distribuer la charge** sur tous les participants, réduisant ainsi la dépendance vis-à-vis d'un point central.
- Les pairs doivent localiser d'autres nœuds pour initier des échanges, en utilisant des algorithmes de découverte ou des protocoles distribués comme les DHT (Distributed Hash Tables).

Systeme Distribué : Architecture

2. Architecture Peer-to-Peer (P2P)

- L'absence de contrôle central rend difficile la gestion de la sécurité et des données.
- La communication peut être plus lente si les pairs sont dispersés géographiquement, et la synchronisation des données peut devenir complexe.
- Des pairs malveillants peuvent entrer facilement dans le réseau, ce qui nécessite des mécanismes de protection avancés.

Systeme Distribué : Architecture

3. Architecture par Microservices

- C'est un modèle d'architecture logicielle dans lequel une application est divisée en **petits services indépendants** et **autonomes**, appelés **micro-services**.
- Chaque micro-service se concentre sur une **fonction** ou un **domaine** spécifique de l'application (comme la gestion des utilisateurs, la facturation, ou les notifications), et les services communiquent entre eux via des API.

Systeme Distribué : Architecture

3. Architecture par Microservices : Avantages

- **Flexibilité** : Permet aux équipes de développer et déployer indépendamment, ce qui réduit le cycle de développement.
- **Résilience** : Les pannes d'un service n'affectent pas nécessairement l'ensemble de l'application.
- **Technologies Hétérogènes** : Chaque service peut utiliser des langages, frameworks et bases de données différents, permettant d'optimiser chaque service en fonction de ses besoins.

Systeme Distribué : Architecture

3. Architecture par Microservices : Inconvénients

- **Complexité Opérationnelle** : Gérer de nombreux services indépendants, chacun avec sa propre base de données, peut être complexe et nécessite des compétences techniques avancées en déploiement et en gestion des systèmes distribués.
- **Gestion des Communications** : La communication entre services peut engendrer une latence et des problèmes de réseau, surtout lorsque les micro-services sont géographiquement dispersés.

Systeme Distribué vs. Parallèle

1. Définition

- **Calcul Distribué** : Un modèle où plusieurs ordinateurs (ou nœuds) interconnectés travaillent ensemble pour accomplir une tâche. Chaque nœud possède sa propre mémoire et ses propres ressources, et ils communiquent via un réseau. Le calcul distribué est souvent utilisé pour des applications nécessitant une gestion de données à grande échelle et une tolérance aux pannes.
- **Calcul Parallèle** : Ce modèle implique la division d'une tâche en sous-tâches qui peuvent être exécutées simultanément sur plusieurs unités de traitement (CPU ou GPU). Les unités de traitement partagent souvent la mémoire, ce qui facilite la communication entre elles.

Systeme Distribué vs. Parallèle

2. Architecture

- **Calcul Distribué** : Les nœuds peuvent être géographiquement dispersés, fonctionnant sur des machines physiques distinctes. L'architecture est souvent plus complexe, car elle nécessite des protocoles de communication pour gérer la distribution des données et des tâches.
- **Calcul Parallèle** : Les unités de traitement travaillent souvent dans un environnement local (comme un seul serveur multi-cœurs), et l'architecture est généralement plus simple. Les threads ou processus partagent souvent une mémoire commune, ce qui réduit le besoin de communication complexe.

Systeme Distribué vs. Parallèle

3. Communication

- **Calcul Distribué** : La communication entre nœuds se fait par le réseau, ce qui peut introduire une latence. Les systèmes distribués doivent gérer les échecs de communication et la synchronisation des données.
- **Calcul Parallèle** : Les communications sont souvent plus rapides, car elles se font via des appels de fonction locaux ou des accès à la mémoire partagée. Cela permet des performances plus rapides pour les calculs qui peuvent être parallélisés efficacement.

Systeme Distribué vs. Parallèle

4. Objectifs

- **Calcul Distribué** : Il est souvent utilisé pour traiter de grandes quantités de données, comme dans le big data, le cloud computing et les systèmes de fichiers distribués. Son objectif est d'améliorer la scalabilité et la tolérance aux pannes.
- **Calcul Parallèle** : Il se concentre sur l'accélération des calculs en tirant parti des ressources de calcul disponibles. Il est utilisé dans des applications nécessitant une puissance de calcul intensive, comme le traitement d'image, les simulations scientifiques et les analyses numériques.



Hadoop 1



Hadoop : Définition

- **Hadoop** est un Framework « open source » de la fondation Apache, qui permet de **stocker** et **traiter** les données massives sur un cluster.
- Il repose sur deux composantes majeurs:
 - **HDFS** (*Hadoop Distributed File System*) : un système de fichier distribués sur un ensemble de nœuds (ou serveur) mais manipulés et vus de l'utilisateur comme un seul fichier.
 - **MapReduce** : paradigme de calcul distribué.

Hadoop : HDFS

1. Généralité:

- **HDFS** est un système de fichiers distribué conçu pour stocker de très gros volumes de données sur un grand nombre de machines.
- Les utilisateurs manipulent les données comme si elles étaient stockées sur un unique ordinateur grâce à la couche d'abstraction fournie par HDFS.
- Dans un système distribué comme HDFS, chaque nœud d'un cluster correspond à un sous-ensemble du volume global de données du cluster. Pour augmenter ce volume global, il suffira d'ajouter de nouveaux nœuds.
- Réplication des données pour assurer la fiabilité et la tolérance aux pannes.

Hadoop : HDFS

2. Fonctionnement:

- Les fichiers sont physiquement découpés en blocs d'octets de grande taille pour optimiser les temps de transfert et d'accès ;
- Par défaut, la taille est fixée à 64 Mo. Il est toutefois possible de monter à 128 Mo, 256 Mo, 512 Mo voire 1 Go.
- Selon la taille d'un fichier, il lui faudra un certain nombre de blocs. le dernier bloc d'un fichier fait la taille restante.
- Ces blocs sont ensuite répartis sur plusieurs machines, permettant ainsi de traiter un même fichier en parallèle.

Hadoop : HDFS

2. Fonctionnement : (suite)

- La répartition de bloc permet aussi de ne pas être limité par la capacité de stockage d'une seule machine pour au contraire tirer parti de tout l'espace disponible du cluster de machines.
- Enfin, pour garantir une tolérance aux pannes, les blocs de chaque fichier sont répliqués, de manière intelligente, sur plusieurs machines.
- Par défaut, chaque bloc est copié sur 3 machines différentes (c'est configurable).
- Chaque fichier se trouve donc en plusieurs exemplaires et à différents endroits.

Hadoop : HDFS

3. Architecture des machines:

Dans Hadoop, l'architecture de stockage est une architecture maître-esclave. Un cluster HDFS est constitué de machines jouant différents rôles exclusifs entre eux :

1. NameNode :

- C'est un service central qui s'occupe de gérer l'état du système de fichiers (C'est le nœud maître du HDFS).
- Il maintient **l'arborescence du système de fichiers** et les **métadonnées** de l'ensemble des fichiers et répertoires d'un systèmes Hadoop.

Hadoop : HDFS

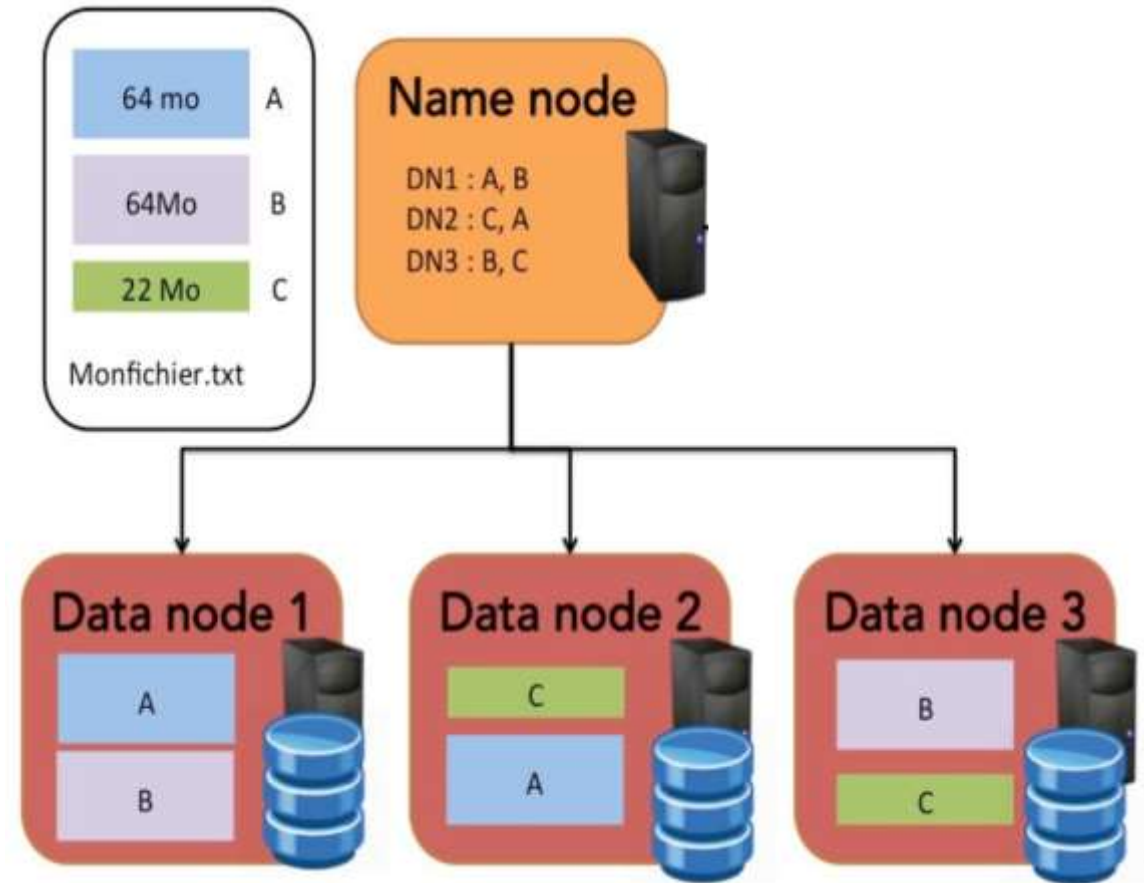
3. Architecture des machines : (suite)

2. DataNodes :

- Nœuds esclaves du Cluster (*il y a un DataNode pour chaque machine pour stocker les données*).
- Dédiés à la gestion des opérations de stockage locales (*manipulation, suppression et réplication de blocs*) sur instruction du name node.
- Ils sont sous les ordres du NameNode. Ils sont sollicités par ce dernier lors des opérations de lecture/écriture.

Hadoop : HDFS

- Les **datanodes** contiennent des blocs. Les mêmes blocs sont dupliqués *sur différents* datanodes, en général 3 fois. Cela assure :
 - fiabilité des données en cas de panne d'un datanode,
 - accès parallèle par différents processus aux mêmes données.
- Le **namenode** sait à la fois :
 - sur quels blocs sont contenus les fichiers,
 - sur quels datanodes se trouvent les blocs voulus.



Hadoop : HDFS

4. Ecriture d'un Fichier dans HDFS:

- Si on souhaite **écrire un fichier dans HDFS**, on utilise un **client Hadoop** (*une ou plusieurs machines jouant le rôle d'un point d'accès au cluster pour s'y connecter et travailler*).
- Le fichier sera divisé en bloc de 64 Mo (*ou autre selon la configuration*). Il va ensuite annoncer au NameNode : « je souhaite stocker ce fichier au sein de HDFS ».
- Le NameNode va alors indiquer qu'il faut stocker le bloc 1 sur le DataNode 1 et le bloc 2 sur le DataNode 3 (par EX).

Hadoop : HDFS

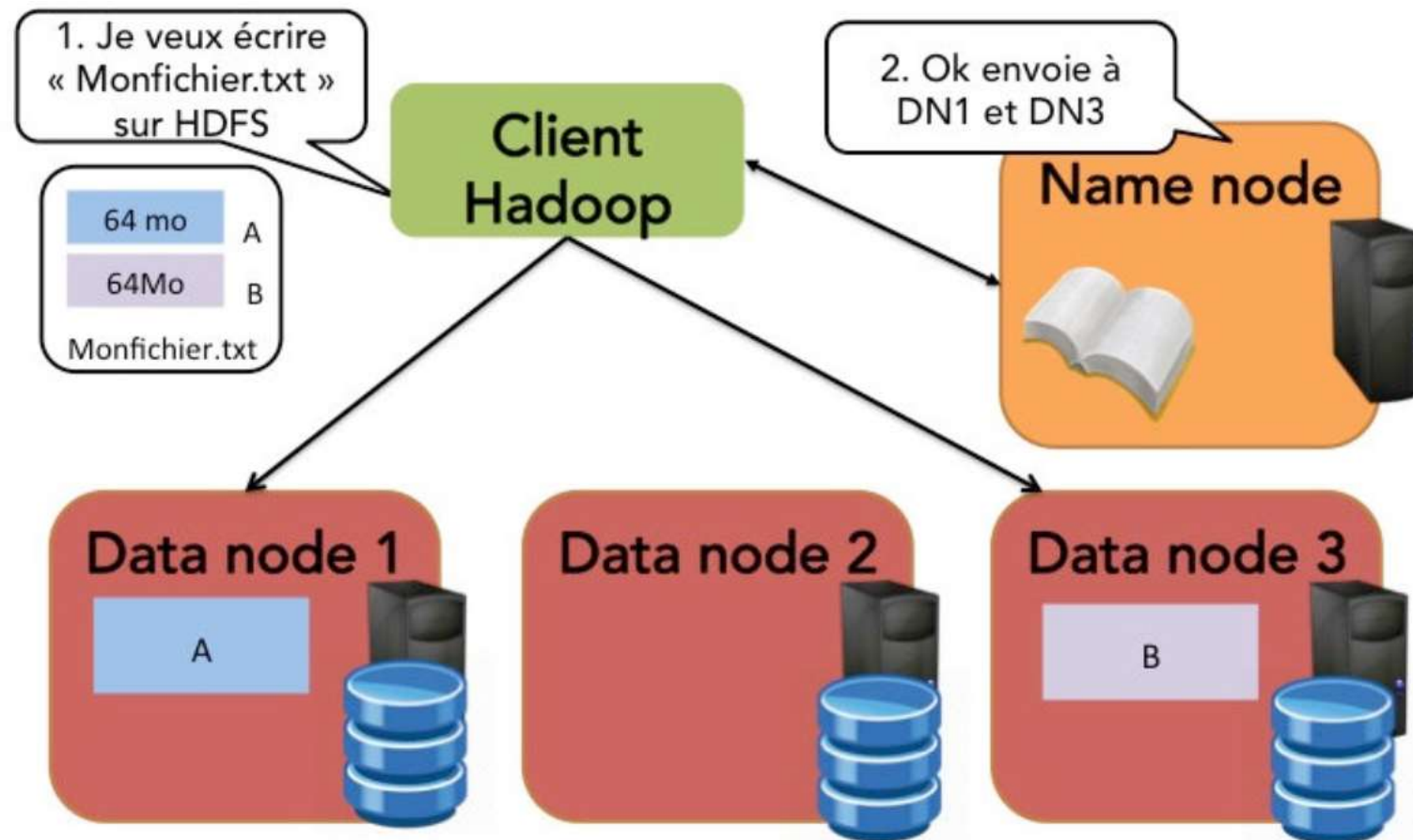
4. Ecriture d'un Fichier dans HDFS: (suite)

Le principe est assez simple :

1. Le client indique au name node qu'il souhaite écrire un bloc.
2. Le name node indique le data node à contacter.
3. Le client envoie le bloc au data node.
4. Les data nodes répliquent les blocs entre eux.
5. Le cycle se répète pour le bloc suivant.

Hadoop : HDFS

4. Ecriture d'un Fichier dans HDFS : (suite)



Hadoop : HDFS

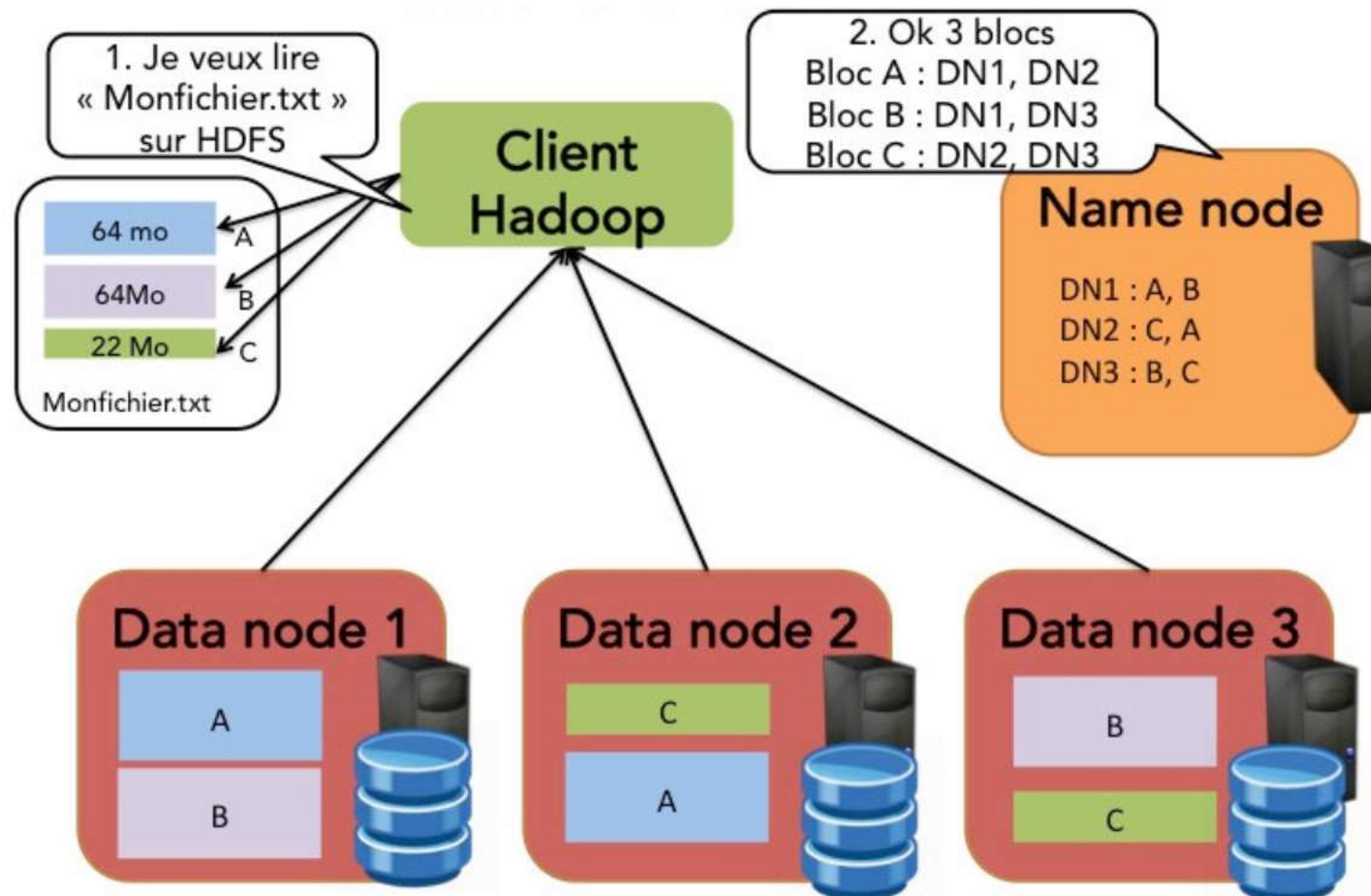
5. Lecture d'un Fichier :

Si on souhaite **lire un fichier dans HDFS**, c'est également assez simple.

1. Le client indique au name node qu'il souhaite lire un fichier.
2. Le name node indique sa taille ainsi que les différents data nodes contenant les blocs.
3. Le client récupère chacun des blocs sur l'un des data nodes.
4. Si le data node est indisponible, le client en contacte un autre.

Hadoop : HDFS

5. Lecture d'un Fichier :



Hadoop : MapReduce

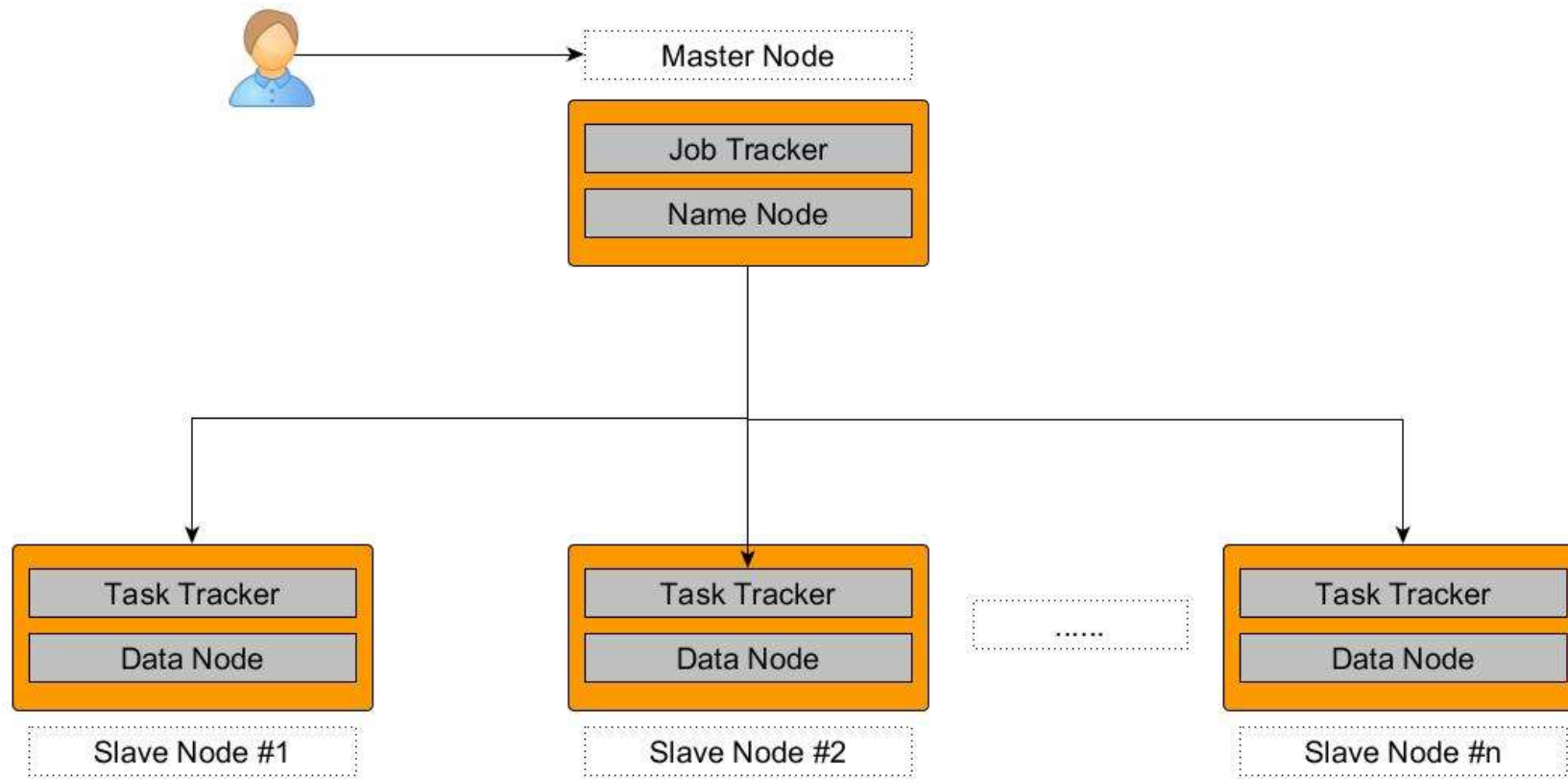
Un « *job* » **MapReduce** : est une unité de travail que le client souhaite réaliser. Cette unité est constituée de:

- **Données d'entrée** (contenues dans HDFS),
- **Programme MapReduce** (implémentation des fonctions map et reduce).
- **Emplacement de stockage du résultat.**

Hadoop exécute ce job en le subdivisant en deux tâches : les tâches de Map et les tâches de Reduce.

Hadoop : Architecture

Comme pour HDFS, la gestion des tâches de Hadoop se base sur deux principaux processus: *Job Tracker* et *Task Tracker*



Hadoop : Architecture

1. Le « *Job Tracker* » :

- Processus maître qui va se charger de **l'ordonnancement des traitements** et de la **gestion de l'ensemble des ressources du système**.
- Il reçoit la tâche à exécuter (*un .jar Java*), ainsi que les données d'entrées (*nom des fichiers stockés sur HDFS*) et le répertoire où stocker les données de sortie (*toujours sur HDFS*).
- Le JobTracker est en communication avec le NameNode de HDFS et sait donc où sont les données.
⇒ Donc, il peut facilement déterminer les meilleures machines auxquelles attribuer les sous-tâches (*celles où les blocs de données correspondants sont stockés*).

Hadoop : Architecture

2. Le « *Task Tracker* » :

- Il y a un *TaskTracker* sur chaque machine du cluster qui **assure l'exécution des tâches MAP/REDUCE s'exécutant sur son nœud.**
- Il est en communication constante avec le *JobTracker* pour recevoir les opérations à effectuer (MAP/REDUCE) ainsi que les blocs de données correspondants (stockés sur HDFS).
- Il envoie aussi les rapports d'avancement au job tracker.

Hadoop 1 : Déroulement de l'exécution d'une tâche

1. Le client (*un outil Hadoop console*) va soumettre le travail à effectuer au **JobTracker** :
 - une archive java .jar implémentant les opérations Map et Reduce,
 - le nom des fichiers d'entrée,
 - l'emplacement où stocker le résultat.
2. Le **JobTracker** communique avec le **NameNode** HDFS pour savoir où se trouvent les blocs correspondants aux données d'entrée.
3. Comme il sait où sont situés les blocs de données, il détermine alors quels sont les nœuds **TaskTracker** les plus appropriés pour exécuter les traitements => *Plannification des traitements*.

Hadoop 1 : Déroulement de l'exécution d'une tâche

4. Il envoie aux **TaskTrackers** sélectionnés (*pour chaque bloc de données*) le travail à effectuer (*Map ou Reduce*).
5. Le **JobTracker** communique avec les noeuds **TaskTracker** en train d'exécuter les tâches:
 - Ils envoient régulièrement un « *heartbeat* », un message signalant qu'ils travaillent toujours sur la sous-tâche reçue.
 - Si aucun *heartbeat* n'est reçu dans une période donnée, le **JobTracker** considère la tâche comme ayant échouée.

Hadoop 1 : Déroulement de l'exécution d'une tâche

6. Si une tâche échoue (*erreur java, données incorrectes, etc.*), le **TaskTracker** va signaler au **JobTracker** que la tâche n'a pas pu être exécutée.

⇒ Dans ce cas, le **JobTracker** va alors décider de la conduite à adopter :

- Demander au même TaskTracker de ré-essayer.
- Redonner la sous-tâche à un autre TaskTracker.
- Marquer les données concernées comme invalides, etc.
- Il pourra même blacklister le TaskTracker concerné comme non-fiable dans certains cas.

Hadoop 1 : Déroulement de l'exécution d'une tâche

7. Une fois que toutes les opérations envoyées aux *TaskTracker* (Map + Reduce) ont été effectuées et confirmées comme effectuées par tous les noeuds, le *JobTracker* marque la tâche comme « effectuée ».

⇒ Vers la fin de l'exécution, des informations détaillées sont disponibles sur le JobTracker (statistiques, TaskTracker ayant posé problème, etc.).

Hadoop 1 : Déroulement de l'exécution d'une tâche

Remarques sur les « TaskTracker » :

- Chaque *TaskTracker* est configuré avec un ensemble de « **Slots** » d'exécution qui indiquent le nombre de tâches qu'il peut accepter.
- Lorsque le *JobTracker* essaie de trouver un emplacement pour planifier une tâche dans les opérations MapReduce, il recherche un slot (emplacement) vide sur le même serveur qui héberge le *DataNode* contenant les données.
- Les « heartbeats » envoyés régulièrement au *JobTracker* par le *TaskTracker* lors du démarrage d'une tâche, indiquent également le nombre de slots disponibles sur le *TaskTracker* concerné.

Hadoop 1 : Architecture Générale

- Généralement, on place le **JobTracker** et le **NameNode** HDFS sur la même machine (*une machine plus puissante que les autres*) communément appelée le **noeud maître** (Master Node).
- Les autres noeuds (*contenant TaskTracker + DataNode*) sont communément appelés **noeuds esclaves** (Slave Node)
- Tout changement de configuration Hadoop peut s'effectuer facilement en changeant la configuration sur la machine où sont situés les serveurs NameNode et JobTracker: ils répliquent les changements de configuration sur tout le cluster automatiquement.

Hadoop 1 : Limitations

1. En pratique, **on ne peut pas résoudre tous les problèmes avec MapReduce**. Et si c'est le cas, cela demande beaucoup d'efforts de transformer un algorithme en MapReduce.
2. **Les slots (CPU, RAM) fixes configurés dans les noeuds**, fait que la gestion de ressources est en manque d'efficience.
3. **Limité à 4000 nœuds maximum par cluster**.
4. **Chaque cluster est géré par un seul JobTracker** : s'il tombe en panne, le cluster tout entier ne peut plus effectuer de tâches.
5. **Un seul Name Node** : une panne du name node introduit la mort de HDFS.

Hadoop 2



Hadoop 2 : Principaux apports

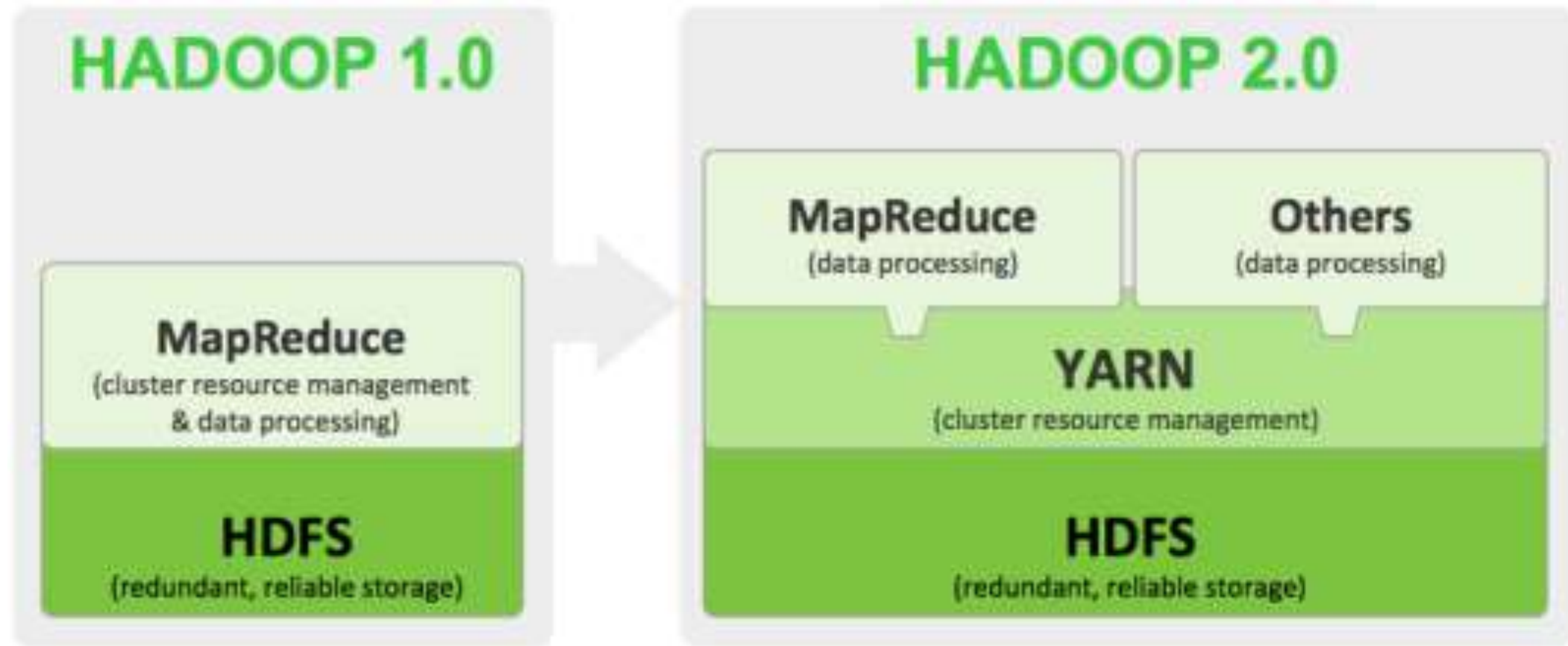
Apparition de **YARN** comme gestionnaire de ressources du cluster, pour remplacer les composants JobTracker et TaskTracker, dans le but de surmonter les limites de la version 1 d'Hadoop.

1. **Implémenter l'algorithme MapReduce en utilisant YARN.**
2. Exécuter n'importe quel type d'application distribuée sur un cluster Hadoop, **pas uniquement les applications MapReduce.**
3. **Séparer la gestion des ressources du cluster et le traitement des données.**

Hadoop 2 : Principaux apports

4. **Améliorer la scalabilité** : jusqu'à 10000 nœuds par cluster
5. **Ajouter un Name Node Secondaire** (*Standby Name Node*):
 - Name node de secours en cas de défaillance du nœud maître.
 - *Rôle* : effectuer des sauvegardes régulières de l'annuaire pour assurer la continuité du fonctionnement du cluster en cas de panne du NameNode principal.
6. **Pas de slots fixes** : Gérer dynamiquement les ressources sur les nœuds en fonction de disponibilités, ce qui augmente l'efficacité de l'utilisation des ressources (via des **containers**)

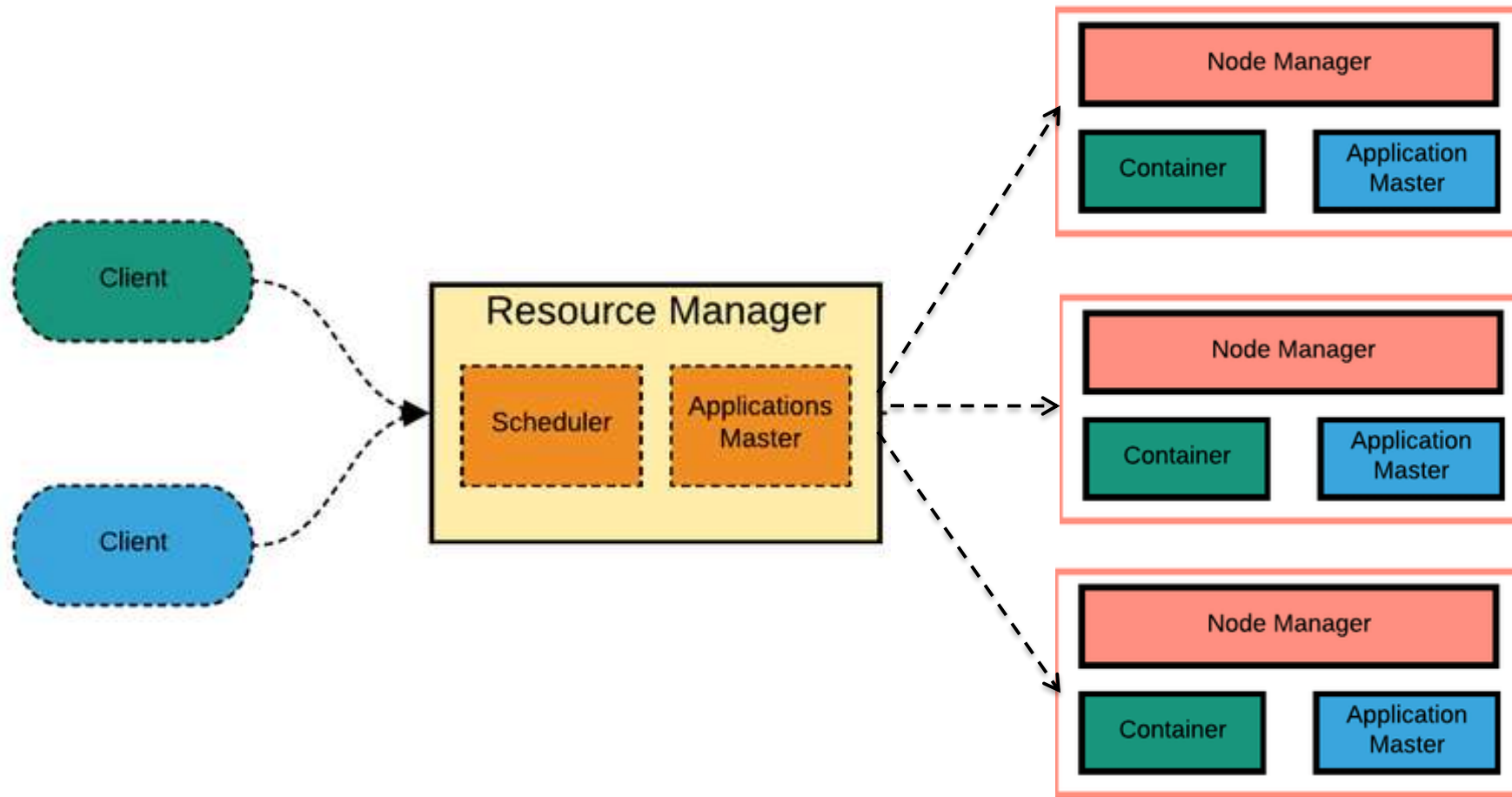
Hadoop 2 : Principaux apports



Hadoop 2 : YARN

- **YARN** (*Yet Another Resource Negotiator*) aussi appelé **MRv2** (**MapReduce 2**). Ce n'est pas une refonte mais une évolution du Framework Mapreduce.
- Il a repenser l'architecture du **JobTracker** :
 - Séparer la **gestion des ressources du cluster** de la **coordination des jobs** → Décharger le JobTracker : ses rôles ont été répartis sur 2 nouveaux job (**JobTracker = ResourceManager + ApplicationMaster**).
 - Utiliser les nœuds esclaves pour gérer les jobs.

Hadoop 2 : YARN (Architecture)



Hadoop 2 : YARN (Architecture)

1. **Resource Manager (RM)** : un *gestionnaire de ressources globale* du cluster. Il n'a maintenant plus que deux tâches bien distinctes à accomplir :
 - **ApplicationsManager** accepte les soumissions d'applications.
 - **Scheduler** est le responsable de allocation des ressources des applications tournant sur le cluster. Ces ressources sont dites *Containers* : un regroupement de mémoire, de CPU, d'espace disque...
 - ⇒ L'idée principale est de considérer que les nœuds ont des ressources (mémoire et CPU) qui seront allouées aux applications quand elles le demandent.

Hadoop 2 : YARN (Architecture)

2. Node Manager :

- C'est un agent tournant sur chaque nœud du cluster
- **Surveille l'utilisation des ressources** (CPU, mémoire) sur le nœud.
- Il **héberge des containers** et gère donc les ressources du nœud.
- Il communique régulièrement avec le ResourceManager (Scheduler), en lui envoyant des rapports d'état sur les ressources disponibles.

Hadoop 2 : YARN (Architecture)

3. **Application Master** : est un processus s'exécutant sur toutes les machines esclaves. Il est chargé de:
- Négocier les ressources (*Containers*) nécessaires auprès du *Scheduler* et en les surveillant via les *NodeManagers*.
 - Lancer (ou relancer au besoin) un Job => *il remplit le rôle du TaskTracker*.
 - Superviser l'état et la progression des jobs.

les *ApplicationMaster* du cluster sont gérés par les *ApplicationsManager* afin de superviser leurs états et les relancer en cas de besoin.

Hadoop 2 : Déroulement d'exécution d'une tâche

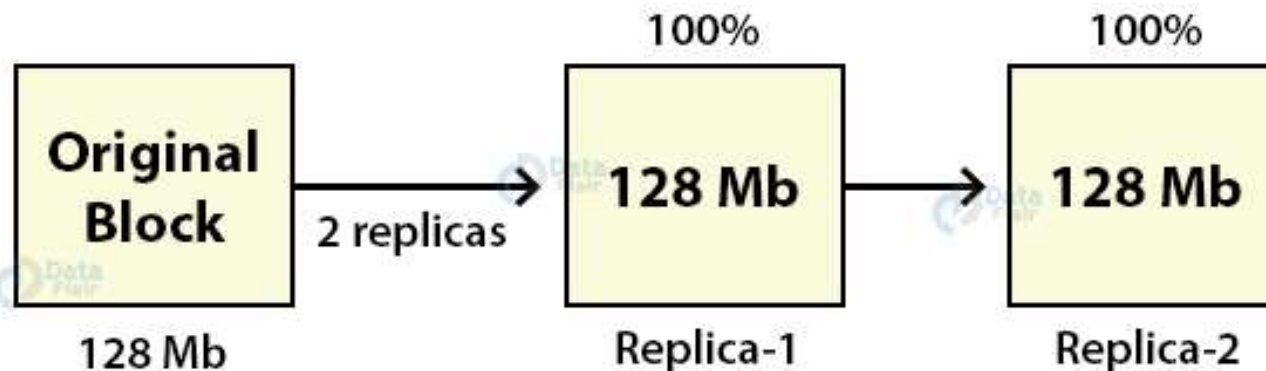
1. Le client va soumettre le travail à effectuer au **RessourceManager** sous la forme d'une *archive.jar* et des noms des fichiers d'entrée et de sortie.
2. Le **RessourceManager** (**Scheduler** plus précisément) alloue un ou plusieurs container(s) à *l'ApplicationMaster* avec des préférences de localisation selon la localité des données d'entrée du travail.
3. **ApplicationMaster** démarre une instance de la tâche choisie dans l'un des containers qui lui a été alloué. Il collabore alors avec le **NodeManager** pour utiliser les ressources acquises. Il communique aussi souvent avec le **RessourceManager** (message *heartbeat*) pour la tolérance aux pannes.

Hadoop 2 : Déroulement d'exécution d'une tâche

4. Parallèlement, chacun des *NodeManager* communique en permanence avec le *RessourceManager* pour lui indiquer son statut en terme de ressources (*containers lancés, RAM, CPU*).
5. A l'issue de l'exécution du programme, l'*ApplicationMaster* s'arrête; son container est libéré et est à nouveau disponible pour de futures tâches.

Hadoop 2 : Limitations

1. **Scalabilité limitée** : 10000 nœuds dans un cluster.
 2. **Admet un seul SBNN.**
 3. **La réplication augmente la surcharge de stockage et semble coûteuse.**
- ⇒ Chaque réplica utilise une surcharge de stockage de 100%, ce qui entraîne une surcharge de stockage de 200%.



Hadoop 3



Hadoop 3 : Principaux apports

1. **Scalabilité améliorée:** peut avoir plus que 10000 nœuds dans un cluster.
2. Supporte la configuration de **2 SBNN ou plus** pour assurer une meilleure disponibilité du cluster.
3. La réplication coûteuse (*surcharge de 200%*) est remplacé par la méthode « **Erasure Coding** » (*codage d'effacement*) qui peut fournir le même niveau de tolérance aux pannes et ne nécessite que **50%** de surcharge de stockage.

Hadoop 3 : Erasure Coding

- Le codage d'effacement (EC) est une branche de la théorie de l'information permettant de **restituer les données manquantes à partir** des **données restantes et de données supplémentaires**, dites de *redondance*.
- Le système de redondance utilisé dans EC est le **calcul de parité**.
- Le codage d'effacement utilise des algorithmes pour calculer la parité pour chaque groupe de données : ***XOR*** et ***Reed-Solomon***.
- Avec le codage d'effacement (Erasure Coding), un fichier avec six blocs ne consommera que **9 blocs d'espace disque** (6 données, 3 parités). Ne nécessite donc que **50% de surcharge de stockage**.

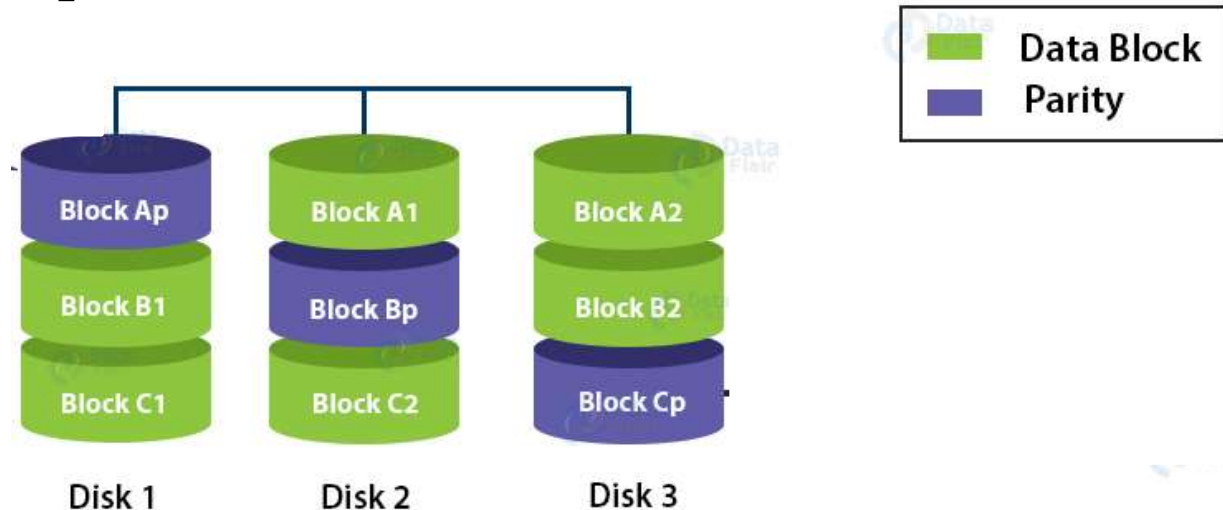
Hadoop 3 : Erasure Coding

– Pour chaque bande (*groupe de bloc*) de l'ensemble de données d'origine, un certain nombre de cellules de parité sont calculées en fonction de l'algorithme de codage d'effacement.

⇒ Ce processus est connu sous le nom de **Codage**.

– Une cellule perdue peut être reconstruite à partir du calcul basé sur les données restantes et les cellules de parité.

⇒ Ce processus est connu sous le nom de **Décodage**



Hadoop : Ecosystème

Comme expliqué précédemment, Hadoop est un système distribué dédié pour le traitement de jeux de données volumineux.

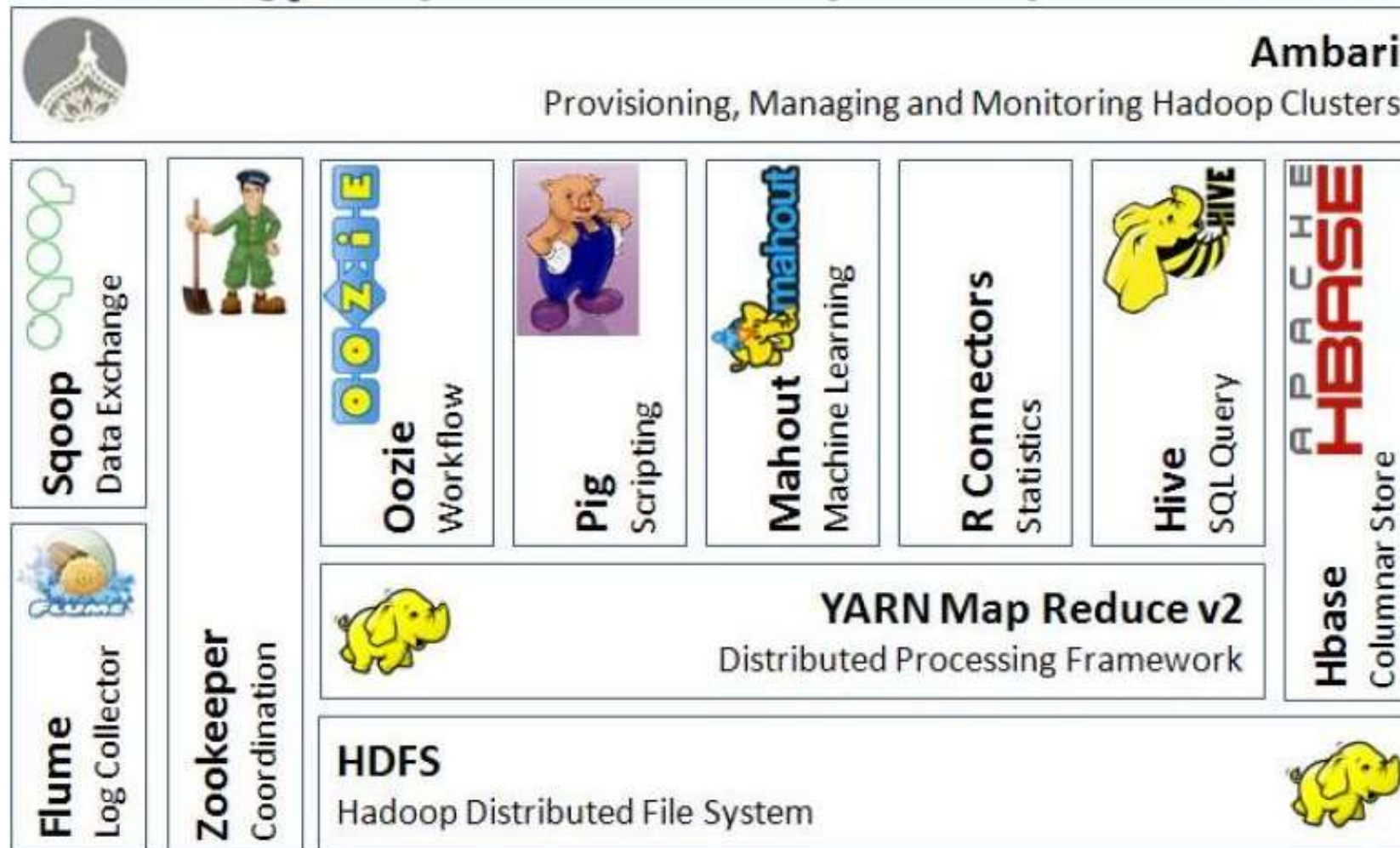
❖ **Problématique :**

Les utilisateurs d'Hadoop se retrouvent face à la manipulation de système de fichiers HDFS ou à développer des programmes MapReduce bas niveau en partant souvent de rien.

✓ **Solution :**

Des sous-projets à Hadoop sont nés de ce constat et offrent des mécanismes et fonctionnalités qui simplifient la manipulation et le traitement des jeux de données volumineux.

Hadoop : Ecosystème



Hadoop : Ecosystème

- **Hbase** : permet l'intégration à Hadoop d'un système de gestion de base de données NoSQL orienté colonnes. Il est distribué, scalable et dédié au Big Data avec un accès direct et une lecture/écriture temps réel.
- **Hive** : son but majeur est de fournir un langage d'abstraction des programmes MapReduce pour minimiser le temps et l'effort de développement. Le projet permet aux développeurs d'écrire des requêtes, dans un langage proche de SQL appelé HiveQL, qui sont ensuite traduites comme des programmes MapReduce sur le cluster.

Hadoop : Ecosystème

- **Pig** : Le projet se positionne comme **Hive** dans le sens où il fournit aux développeurs un langage d'abstraction de haut niveau (un DSL) dédié à l'analyse de gros volumes de données. Il s'adresse alors aux développeurs habitués à faire des scripts via Bash ou Python par exemple.

Dans le même ordre d'idées que le projet **Pig**, il y a **Scalding** qui puise la puissance du langage Scala pour développer ses programmes MapReduce.

Hadoop : Ecosystème

- **Sqoop** : est un outil conçu spécifiquement pour transférer des données entre Hadoop (HDFS) et des bases de données relationnelles. Sqoop permet également d'exporter des données depuis Hadoop vers des bases de données relationnelles.
- **Mahout** : est une bibliothèque d'apprentissage automatique distribuée conçue pour faciliter le développement et le déploiement d'applications d'analyse prédictive à grande échelle sur la plateforme Hadoop. Il implémente des algorithmes de classification et de regroupement automatique (Machine Learning, DataMining) dans un environnement distribué.

Hadoop : Ecosystème

- **Flume** : Flume est un outil conçu pour l'ingestion de données en continu (streaming) dans Hadoop.

Il est principalement utilisé pour collecter et déplacer des données de manière fiable et efficace à partir de sources de données variées vers Hadoop HDFS.

Hadoop : Distribution



Manipulation de l'HDFS



HDFS : Méthodes de Manipulations

Il y a deux possibilités pour manipuler l'HDFS :

- Soit via l'*API Java*, que nous ne décrirons dans les séries de TP.
- Soit directement depuis un terminal via les *commandes*.

Manipulation HDFS : Commandes

Commande	Fonctionnalité
hdfs dfs -help	Afficher l'aide
hdfs dfs -put <local_src> <hdfs_dest>	Copier un fichier local dans l'hdfs
hdfs dfs -copyFromLocal <local_src> <hdfs_dest>	
hdfs dfs -get <hdfs_dest> <local_src>	Copier un fichier à partir de l'hdfs sur votre disque local
hdfs dfs -copyToLocal <hdfs_dest> <local_src>	
hdfs dfs -moveFromLocal <local_src> <hdfs_dest>	Déplacer un fichier local vers l'hdfs
hdfs dfs -moveToLocal <hdfs_dest> <local_src>	Déplacer un fichier de l'hdfs vers le local

Manipulation HDFS : Commandes

Commande	Fonctionnalité
hdfs dfs -cat <hdfs_path_file>	Afficher tout le contenu d'un fichier
hdfs dfs -tail <path_file>	Afficher en console le dernier 1KB du fichier
hdfs dfs -mv <hdfs_file_src> <hdfs_dest>	Déplacer un fichier d'une source vers une destination
hdfs dfs -rm file_or_rep_path	Supprimer un fichier ou un répertoire
hdfs dfs -cp <hdfs_file_src> <hdfs_dest>	Copier un fichier d'un répertoire à un autre
hdfs dfs -mkdir [-p] /hdfs_path/rep_name	Créer un répertoire dans l'hdfs <i>-p : pour créer plusieurs répertoires en même temps</i> <i>/hdfs_path/r1/r2/r3</i>
hdfs dfs -ls [-R] [répertoire]	Afficher le contenu d'un répertoire <i>-R : pour lister le contenu en détail avec les sous répertoires</i>

Manipulation HDFS : Commandes

Récapitulatif général:

- Toutes les commandes interagissant avec le système Hadoop commencent par **hadoop fs** (aussi **hdfs dfs**).
- Ensuite, les options rajoutées sont très largement inspirées des commandes Unix standard.

Pour plus d'information sur les commandes HDFS, consulter la [documentation](#).